แกะรอยวิธีคิด LLM: จากคำสั่ง (Prompt) สู่ข้อความที่เราอ่าน

1. บทนำ: LLM คืออะไร และมัน "คิด" อย่างไร?

หลายคนอาจเคยได้ยินคำว่า LLM (Large Language Model) หรือแบบจำลองภาษาขนาดใหญ่ และสงสัยว่ามันคืออะไรกันแน่ มันมีเวทมนตร์อะไรที่ทำให้สามารถตอบคำถาม พูดคุย หรือแม้กระทั่งเขียนบทความได้ราวกับมนุษย์ วันนี้เราจะมาไขความลับเบื้องหลังความสามารถอันน่าทึ่งนี้กัน

LLM คืออะไร? ไม่ใช่เวทมนตร์ แต่เป็นคณิตศาสตร์อันชาญฉลาด!

เริ่มต้นจากการทำความเข้าใจกันก่อนว่า LLM ไม่ได้ "คิด" แบบเดียวกับที่มนุษย์คิด [1] มันไม่ใช่สิ่งมีชีวิตที่มีความรู้สึกนึกคิดหรือความเข้าใจโลกอย่างแท้จริง แต่เป็นโปรแกรมคอมพิวเตอร์ที่ซับซ้อนอย่างยิ่ง ซึ่งเรียนรู้จากข้อมูลตัวอักษรจำนวนมหาศาล

ลองจินตนาการว่า LLM เป็นเหมือน "สุดยอดระบบเติมคำอัตโนมัติ" ที่เราใช้บนโทรศัพท์มือถือ แต่มีความสามารถสูงกว่ามาก [1] หรือเปรียบได้กับ "นกแก้วผู้รอบรู้" ที่ได้อ่านหนังสือและบทความเกือบทั้งหมดบนอินเทอร์เน็ตและในห้องสมุด [1] มันเก่งกาจอย่างเหลือเชื่อในการ "ทาย" ว่าคำใดควรจะปรากฏเป็นลำดับถัดไปในประโยค โดยอ้างอิงจากรูปแบบและความสัมพันธ์ของคำที่มันเคย "อ่าน" หรือ "เรียนรู้" มา [1] "ความรู้" ของ LLM จึงเป็นเรื่องของรูปแบบทางสถิติเหล่านี้ ไม่ใช่ความเข้าใจเชิงลึกหรือสติปัญญาแบบมนุษย์

การที่ LLM สามารถสร้างข้อความที่ดูเหมือนฉลาดและมีความรู้ได้นั้น เป็นเพราะมันถูกฝึกฝนด้วยข้อมูลจำนวนมหาศาล ทำให้มันสามารถจับรูปแบบที่ซับซ้อนในภาษาได้ อย่างไรก็ตาม หากข้อมูลที่ใช้ฝึกฝนมีข้อผิดพลาดหรือความลำเอียง LLM ก็อาจจะแสดงผลลัพธ์ที่ผิดพลาดหรือลำเอียงเหล่านั้นออกมาได้เช่นกัน เหมือนกับนกแก้วที่อาจจะพูดสิ่งที่มันได้ยินมาผิดๆ โดยไม่รู้ตัว [1] สิ่งนี้อธิบายได้ว่าทำไมบางครั้ง LLM ถึงให้ข้อมูลที่ดูน่าเชื่อถือแต่กลับไม่ถูกต้อง หรือที่เรียกว่า "การหลอน" (hallucination) [2]

คำถามสำคัญ: จากคำสั่งของเรา ไปสู่คำตอบของ LLM ได้อย่างไร?

เมื่อเราพิมพ์คำสั่งหรือคำถาม (ที่เราเรียกว่า "Prompt") เข้าไปใน LLM มันมีกระบวนการอะไรเกิดขึ้นบ้างเบื้องหลัง? LLM "อ่าน" คำสั่งของเราอย่างไร? มัน "คิด" เกี่ยวกับคำสั่งนั้นด้วยวิธีพิเศษแบบไหน? และสุดท้าย มัน "เขียน" คำตอบออกมาเป็นประโยคหรือบทความได้อย่างไร?

รายงานฉบับนี้จะพาไปสำรวจการเดินทางของคำสั่งของเรา ตั้งแต่เริ่มต้นจนกระทั่งกลายเป็นผลลัพธ์ที่เราเห็น โดยจะอธิบายแต่ละขั้นตอนอย่างละเอียด เพื่อให้เข้าใจกลไกการทำงานของ LLM ได้อย่างชัดเจนยิ่งขึ้น

2. ขั้นตอนที่ 1: เมื่อคุณพิมพ์คำสั่ง – อะไรเกิดขึ้นก่อน? (การแบ่งเป็นโทเค็น -

Tokenization)

เมื่อเราป้อนคำสั่งหรือ Prompt เข้าไปใน LLM สิ่งแรกที่เกิดขึ้นไม่ใช่การที่ LLM เข้าใจประโยคหรือย่อหน้าทั้งหมดในทันที แต่ LLM จะต้อง "อ่าน" คำสั่งของเราโดยการแบ่งข้อความนั้นออกเป็นชิ้นส่วนเล็กๆ ที่เรียกว่า "โทเค็น" (Tokens) เสียก่อน [3]

โทเค็นคืออะไร?

โทเค็นเหล่านี้อาจเป็นคำทั้งคำ (เช่น "แมว", "นั่ง"), ส่วนของคำ (sub-words) (เช่น คำว่า "น่าสนใจ" อาจถูกแบ่งเป็น "น่า-สน-ใจ") หรือแม้กระทั่งตัวอักษรแต่ละตัว ทั้งนี้ขึ้นอยู่กับการออกแบบของ LLM แต่ละตัว [3]

- ตัวอย่างการแบ่งโทเค็นแบบคำ (Word Tokenization): ประโยค "อธิบาย LLM แบบง่ายๆ" อาจถูกแบ่งออกเป็น ["อธิบาย", "LLM", "แบบ", "ง่ายๆ", "."] [6]
- ตัวอย่างการแบ่งโทเค็นแบบส่วนของคำ (Sub-word Tokenization): คำว่า "การแบ่งโทเค็นมีประโยชน์" อาจถูกแบ่งเป็น ["การ", "แบ่ง", "โท", "เค็น", "มี", "ประ", "โยชน์", "."] [6] วิธีนี้ช่วยให้ LLM สามารถจัดการกับคำศัพท์ใหม่ๆ หรือคำที่พบไม่บ่อยได้ดีขึ้น โดยการแยกคำเหล่านั้นออกเป็นส่วนย่อยที่มันรู้จัก

ลองนึกภาพว่าการแบ่งโทเค็นเหมือนกับการ "หั่นส่วนผสมต่างๆ ให้เป็นชิ้นเล็กๆ ก่อนนำไปปรุงอาหาร" แต่ละคำหรือส่วนของคำก็เปรียบเสมือนส่วนผสมชิ้นเล็กๆ ที่ LLM สามารถจัดการและประมวลผลได้ง่ายขึ้น หรืออาจเปรียบได้กับการแยกประโยคยาวๆ ออกเป็น "ตัวต่อเลโก้" ทีละชิ้น

ทำไมต้องแบ่งเป็นโทเค็น?

การแบ่งข้อความออกเป็นโทเค็นมีประโยชน์หลายประการ:

1. สร้างชุดคำศัพท์ที่แน่นอน: มันช่วยให้ LLM มีชุดคำศัพท์ (vocabulary) ที่จำกัดและจัดการได้ แม้ว่าภาษาจะมีคำศัพท์มากมายไม่สิ้นสุดก็ตาม
2. จัดการกับคำที่ไม่เคยเห็น: ช่วยให้ LLM สามารถรับมือกับคำศัพท์ใหม่ๆ หรือคำที่ไม่เคยพบเห็นมาก่อนในระหว่างการฝึกฝนได้ โดยการแบ่งคำเหล่านั้นออกเป็นส่วนย่อย (sub-words) ที่มันรู้จัก [3]
3. แปลงข้อความเป็นรูปแบบที่เครื่องจักรเข้าใจได้: การแบ่งโทเค็นเป็นการแปลงข้อความที่ไม่มีโครงสร้างให้อยู่ในรูปแบบที่มีโครงสร้าง ซึ่งแบบจำลองการเรียนรู้ของเครื่อง (machine learning model) สามารถเข้าใจและนำไปประมวลผลต่อได้ [4]

กระบวนการแบ่งโทเค็นนี้ถือเป็นด่านแรกที่ "ความหมาย" จากคำสั่งของมนุษย์เริ่มถูกแปลให้อยู่ในรูปแบบที่เครื่องจักรสามารถทำความเข้าใจได้ กลยุทธ์ในการแบ่งโทเค็น เช่น การเลือกใช้การแบ่งแบบคำ แบบส่วนของคำ หรือแบบตัวอักษร จะส่งผลโดยตรงต่อขนาดของชุดคำศัพท์ของแบบจำลองและความสามารถในการจัดการกับคำที่ไม่คุ้นเคย การแบ่งแบบส่วนของคำ (เช่น Byte Pair Encoding หรือ BPE ที่กล่าวถึงใน [7]) เป็นวิธีการที่ชาญฉลาดที่ช่วยให้ชุดคำศัพท์มีขนาดที่จัดการได้ ในขณะที่ยังคงความสามารถในการแทนความหมายและประมวลผลคำใหม่ๆ หรือคำที่พบได้น้อย [3] นี่

หมายความว่า "หน่วยการสร้าง" ความเข้าใจของ LLM นั้นมีความยืดหยุ่นสูง
สำหรับภาษาไทยซึ่งมีลักษณะเฉพาะคือการเขียนคำต่อเนื่องกันโดยไม่มีการเว้นวรรคระหว่างคำ และมีโครงสร้างตัวอักษรที่แตกต่างจากภาษาอังกฤษ การแบ่งโทเค็นจึงเป็นขั้นตอนที่ท้าทายและมีความสำคัญอย่างยิ่งต่อประสิทธิภาพของ LLM ที่พัฒนาขึ้นสำหรับภาษาไทยโดยเฉพาะ

3. ขั้นตอนที่ 2: เปลี่ยนคำให้เป็นภาษาของ LLM – เอ็มเบดดิ้ง (Embeddings)

หลังจากที่ข้อความถูกแบ่งออกเป็นโทเค็นแล้ว LLM ยังไม่สามารถทำงานกับโทเค็นที่เป็นตัวอักษรเหล่านั้นได้โดยตรง เพราะคอมพิวเตอร์เข้าใจตัวเลข ไม่ใช่คำพูด ดังนั้น แต่ละโทเค็นจะถูกแปลงให้อยู่ในรูปของชุดตัวเลข ที่เรียกว่า "เอ็มเบดดิ้ง" (Embedding) หรือ "เวกเตอร์" (Vector) [3]
คำต้องแปลงเป็นตัวเลข: แนวคิดของเอ็มเบดดิ้ง
ลองจินตนาการว่าเอ็มเบดดิ้งคือการ "กำหนดพิกัดเฉพาะให้กับแต่ละคำบน 'แผนที่ความหมาย' ขนาดใหญ่" คำที่มีความหมายคล้ายกันจะมีพิกัด (หรือเวกเตอร์) ที่คล้ายกัน และจะอยู่ "ใกล้กัน" บนแผนที่นี้ [4] ตัวอย่างเช่น คำว่า "มีความสุข" และ "ร่าเริง" อาจมีพิกัดใกล้เคียงกัน ในขณะที่คำว่า "มีความสุข" และ "เศร้า" จะอยู่ห่างกันออกไป เวกเตอร์เหล่านี้มักจะมีมิติสูง (ประกอบด้วยตัวเลขหลายร้อยหรือหลายพันตัว) ซึ่งสามารถจับแง่มุมความหมายที่ละเอียดอ่อนได้มากมาย [3]
ตัวเลขเหล่านี้ (เวกเตอร์) แทนอะไร? ความหมายเชิงความสัมพันธ์ (Semantic Meaning)
เอ็มเบดดิ้งทำหน้าที่เก็บ "ความหมายเชิงความสัมพันธ์" ของโทเค็นและความเชื่อมโยงระหว่างโทเค็นเหล่านั้น [3] LLM เรียนรู้ความหมายเหล่านี้จากวิธีการใช้คำต่างๆ ในข้อมูลตัวอักษรจำนวนมหาศาลที่ใช้ในการฝึกฝน ตัวอย่างที่มักถูกยกมาคือความสัมพันธ์แบบ "ราชา - ชาย + หญิง = ราชินี" ซึ่งแสดงให้เห็นว่าเอ็มเบดดิ้งสามารถจับความสัมพันธ์ทางความหมายได้ [8] นอกจากนี้ เอ็มเบดดิ้งยังช่วยให้ LLM เข้าใจว่าคำเดียวกันอาจมีความหมายแตกต่างกันไปตามบริบท เช่น คำว่า "bank" อาจหมายถึง "ตลิ่ง" หรือ "ธนาคาร" ก็ได้ [8]
อย่าลืมลำดับคำ! การเข้ารหัสตำแหน่ง (Positional Encoding)
แบบจำลอง Transformer ซึ่งเป็นสถาปัตยกรรมหลักของ LLM ส่วนใหญ่ ประมวลผลคำต่างๆ พร้อมกัน (parallel processing) เพื่อความรวดเร็ว ซึ่งหมายความว่าโดยตัวมันเองแล้ว มันไม่รู้ลำดับของคำในประโยค [5] แต่ลำดับของคำมีความสำคัญอย่างยิ่งต่อความหมาย เช่น "แมวไล่จับหนู" มีความหมายแตกต่างจาก "หนูไล่จับแมว"
เพื่อแก้ปัญหานี้ จึงมีการใช้ "การเข้ารหัสตำแหน่ง" (Positional Encoding) โดยการเพิ่มชุดตัวเลขพิเศษเข้าไปในเอ็มเบดดิ้งของแต่ละโทเค็น เพื่อให้ LLM รู้ข้อมูลเกี่ยวกับตำแหน่งของโทเค็นนั้นๆ ในลำดับของประโยค [5] ลองนึกภาพว่ามันเหมือนกับการ "ใส่หมายเลขกำกับส่วนผสมในสูตรอาหาร เพื่อรักษลำดับขั้นตอนที่ถูกต้อง" หรือ "การประทับเวลา (timestamp) ให้กับแต่ละคำ"
คุณภาพของเอ็มเบดดิ้งมีความสำคัญอย่างยิ่งต่อประสิทธิภาพของ LLM หากเอ็มเบดดิ้งไม่สามารถจับความคล้ายคลึงทางความหมายและความสัมพันธ์ระหว่างคำได้อย่างแม่นยำ ชั้นถัดๆ ไปของ LLM ก็จะประสบปัญหาในการทำความเข้าใจ

ข้อมูลนำเข้า แนวคิดของ "แผนที่ความหมาย" หรือปริภูมิเวกเตอร์ (vector space) ชี้ให้เห็นว่าความหมายไม่ใช่สิ่งที่แยกจากกันอย่างชัดเจน แต่มีความต่อเนื่อง LLM ทำงานในปริภูมิต่อเนื่องนี้ ทำให้สามารถเข้าใจระดับความหมายที่แตกต่างกันและอุปมาอุปมัยได้ในแบบที่ระบบที่อิงตามกฎเกณฑ์ไม่สามารถทำได้ [4]

การเข้ารหัสตำแหน่ง [5] เป็นเหมือน "ส่วนเสริม" ที่ถูกสร้างขึ้นเพื่อชดเชยข้อจำกัดของกลไก Self-Attention ใน Transformer ที่โดยธรรมชาติแล้วไม่ได้รับรู้ถึงลำดับของข้อมูล แม้ว่าจะเป็นวิธีที่มีประสิทธิภาพ แต่ก็แสดงให้เห็นว่าแม้แต่สถาปัตยกรรมที่ก้าวหน้าก็ยังมีข้อจำกัดเฉพาะที่ต้องการวิธีแก้ปัญหาที่ชาญฉลาด สิ่งนี้สะท้อนถึงธรรมชาติของการวิจัยและพัฒนา AI ที่มีการปรับปรุงและพัฒนาอย่างต่อเนื่อง

4. ขั้นตอนที่ 3: ภายใน "สมอง" ของ LLM – Transformer และ Attention (ที่นี่คือที่ที่การ "ค้นหา" เกิดขึ้น)

เมื่อโทเค็นจากคำสั่งของเราถูกแปลงเป็นเอ็มเบดดิ้งและมีการเพิ่มข้อมูลตำแหน่งแล้ว ข้อมูลเหล่านี้จะถูกส่งเข้าไปประมวลผลในส่วนที่เปรียบเสมือน "สมอง" ของ LLM ซึ่งก็คือสถาปัตยกรรมที่เรียกว่า "Transformer" [5]

ห้องเครื่องยนต์: สถาปัตยกรรม Transformer

Transformer เป็นสถาปัตยกรรมหลักที่อยู่เบื้องหลัง LLM สมัยใหม่ส่วนใหญ่ เช่น GPT มันประกอบด้วยชั้น (layers) ที่ทำงานซ้ำๆ กัน (เรียกว่า Encoder และ/หรือ Decoder blocks) โดยแต่ละชั้นจะปรับปรุงและกลั่นกรองข้อมูลที่ได้รับมาจากชั้นก่อนหน้า [5] ลองนึกภาพว่า Transformer เป็นเหมือนโรงงานที่ซับซ้อนซึ่งมีสายการผลิตหลายทอด (ชั้นต่างๆ) โดยแต่ละสายการผลิตทำหน้าที่เฉพาะในการประมวลผลและทำความเข้าใจข้อมูล (เอ็มเบดดิ้งของโทเค็น)

ดาวเด่นของการแสดง: กลไก Attention

หัวใจสำคัญที่ทำให้ Transformer ทรงพลังคือกลไกที่เรียกว่า "Attention" (การใส่ใจ) [12] นี่คือวิธีที่ LLM ตัดสินใจว่าส่วนใดของคำสั่งที่เราป้อนเข้าไป (และส่วนของข้อความที่มันสร้างขึ้นเองก่อนหน้านั้น) มีความสำคัญที่สุดในการทำความเข้าใจบริบทและคิดว่าจะตอบสนองอย่างไรต่อไป มันไม่ใช่การ "ค้นหา" ข้อมูลในฐานข้อมูลเหมือนกับที่ Google ทำ แต่เป็นการคำนวณ "คะแนนความใส่ใจ" (attention scores) ระหว่างโทเค็นต่างๆ

Query, Key, Value (QKV): หัวใจของ Attention [12]

สำหรับแต่ละโทเค็น LLM จะสร้างเวกเตอร์พิเศษขึ้นมา 3 ตัว ได้แก่:

- Query (Q): เปรียบเหมือน "คำถาม" ของโทเค็นปัจจุบัน หรือสิ่งที่มันกำลังมองหา เช่น "ข้อมูลอะไรที่เกี่ยวข้องกับฉัน?" [12]
- Key (K): เปรียบเหมือน "ป้ายกำกับ" ของโทเค็น หรือชนิดของข้อมูลที่มันนำเสนอ เช่น "นี่คือสิ่งที่ฉันเป็น" [12]
- Value (V): เปรียบเหมือน "เนื้อหา" หรือความหมายที่แท้จริงของโทเค็นนั้นๆ เช่น "นี่คือข้อมูลที่ฉันมี" [12]

การทำงานร่วมกันของ Q, K, V:

1. Query ของโทเค็นหนึ่งจะถูกนำไปเปรียบเทียบ (มักใช้การคำนวณ dot product) กับ Key ของโทเค็นอื่นๆ

ทั้งหมด (รวมถึงตัวมันเอง) ในลำดับข้อมูล [15] ผลลัพธ์ที่ได้คือ "คะแนนความคล้ายคลึง" หรือ "คะแนนความใส่ใจ"

2. คะแนนเหล่านี้จะถูกปรับให้เป็นมาตรฐาน (normalized) โดยใช้ฟังก์ชันที่เรียกว่า softmax เพื่อแสดงว่าโทเค็นปัจจุบันควร "ใส่ใจ" กับโทเค็นอื่นๆ แต่ละตัวมากน้อยเพียงใด [15]
3. จากนั้น Value เวกเตอร์ของโทเค็นทั้งหมดจะถูกนำมารวมกัน โดยถ่วงน้ำหนักตามคะแนนความใส่ใจเหล่านี้ ผลลัพธ์ที่ได้คือเวกเตอร์ใหม่สำหรับโทเค็นปัจจุบัน ซึ่งเต็มไปด้วยบริบทจากโทเค็นอื่นๆ ที่เกี่ยวข้อง [15]

อุปมาอุปมัยสำหรับ QKV:

- อุปมัยห้องสมุด (Library Analogy) [17]:
  - Query: คำถามวิจัยของคุณ (เช่น "ไดโนเสาร์")
  - Keys: คำสำคัญบนสันหนังสือ (เช่น "บรรพชีวินวิทยา", "จูแรสซิก", "สัตว์เลื้อยคลาน")
  - Values: ข้อมูลจริงภายในหนังสือเหล่านั้น
  - LLM จะค้นหาหนังสือที่มี Keys ตรงกับ Query ของคุณ แล้วสรุปเนื้อหาจาก Values ของหนังสือเหล่านั้นโดยถ่วงน้ำหนักความสำคัญ
- อุปมัยการจับคู่/ค้นหาผู้เชี่ยวชาญ (Matchmaking/Expert Finding Analogy) [18]:
  - Query: "ฉันต้องการผู้เชี่ยวชาญด้านอาหารอิตาเลียน"
  - Key: โปรไฟล์ของบุคคลที่ระบุว่า "ฉันเป็นผู้เชี่ยวชาญด้านอาหารอิตาเลียน" หรือ "ฉันรู้เรื่องพาสต้า"
  - Value: คำแนะนำหรือสูตรอาหารที่บุคคลนั้นให้
  - LLM จะจับคู่ความต้องการของคุณ (Query) กับความเชี่ยวชาญที่มีอยู่ (Keys) เพื่อให้ได้ข้อมูลที่เป็นประโยชน์ (Values)
- อุปมัยสปอตไลท์ (Spotlight Analogy) [22]: เมื่อประมวลผลคำใดคำหนึ่ง LLM จะส่องสปอตไลท์ไปยังคำอื่นๆ ที่เกี่ยวข้องในประโยค สปอตไลท์ที่สว่างกว่าหมายถึงการใส่ใจที่มากกว่า

Self-Attention กับ Cross-Attention:

- Self-Attention (การใส่ใจตนเอง): โทเค็นภายในลำดับข้อมูล *เดียวกัน* (เช่น เฉพาะในส่วนของคำสั่ง หรือเฉพาะในส่วนของคำตอบที่ LLM สร้างขึ้น) จะใส่ใจซึ่งกันและกันเพื่อสร้างความเข้าใจภายใน [5]
- Cross-Attention (การใส่ใจข้ามส่วน - ในแบบจำลอง Encoder-Decoder): ส่วน Decoder (ที่สร้างคำตอบ) จะใส่ใจกับผลลัพธ์จากส่วน Encoder (ที่ประมวลผลคำสั่ง) นี่คือวิธีที่การสร้างคำตอบถูกชี้นำโดยคำสั่งที่ป้อนเข้ามา [5]

Multi-Head Attention: หลายมุมมอง [5] LLM ไม่ได้ทำกระบวนการ QKV นี้เพียงครั้งเดียว แต่มันมี "หัวการใส่ใจ" (attention heads) หลายหัวที่ทำงานพร้อมกัน โดยแต่ละหัวจะมุ่งเน้นไปที่ความสัมพันธ์หรือแง่มุมที่แตกต่างกันของข้อความ เปรียบเหมือนการมีผู้เชี่ยวชาญหลายคน (หัวต่างๆ) มาพิจารณาคำสั่งของคุณจากมุมมองที่แตกต่างกัน (เช่น ไวยากรณ์ ความหมาย บริบท) แล้วนำข้อมูลเชิงลึกของพวกเขามารวมกัน (อุปมัยคณะผู้เชี่ยวชาญ [22])

การแปลงข้อมูล: สถานะซ่อนเร้น (Hidden States) ขณะที่ข้อมูลของโทเค็น (ซึ่งตอนนี้เป็นเวกเตอร์บริบทที่สมบูรณ์จาก

การทำงานของ attention) ไหลผ่านชั้นต่างๆ ของ Transformer มันจะถูกแปลงซ้ำแล้วซ้ำเล่า การแทนข้อมูลด้วยเวกเตอร์ในระหว่างแต่ละชั้นนี้มักเรียกว่า "สถานะซ่อนเร้น" (hidden states) [14] สถานะซ่อนเร้นเหล่านี้คือ "ความเข้าใจ" หรือ "บทสรุป" ที่กำลังพัฒนาของ LLM เกี่ยวกับข้อมูลนำเข้า และยังคงอยู่ในรูปของเวกเตอร์ มีงานวิจัยที่ชี้ว่าสถานะซ่อนเร้นเหล่านี้สามารถเข้ารหัสข้อมูลเกี่ยวกับความถูกต้องของคำตอบระหว่างการประมวลผลได้ด้วยซ้ำ [23] และชั้นล่างๆ ของแบบจำลองอาจมุ่งเน้นไปที่การรวบรวมข้อมูล ในขณะที่ชั้นบนๆ อาจประมวลผลข้อมูลนั้นภายในตัวเองมากขึ้น [25]

ชี้แจง "เวกเตอร์ หรือ ชิ้นส่วนข้อมูล (Chunk)" – LLM หลักใช้เวกเตอร์

ผู้ใช้สอบถามว่าข้อมูลที่ "ค้นหา" มานั้นเป็น "เวกเตอร์ หรือ ชิ้นส่วนข้อมูล (chunk)"

- ภายในกระบวนการหลักของ Transformer LLM จะจัดการกับเวกเตอร์ (VECTORS) เอ็มเบดดิ้งคือเวกเตอร์ สถานะซ่อนเร้นคือเวกเตอร์ ผลลัพธ์จากการคำนวณ attention ก็คือเวกเตอร์
- "ชิ้นส่วนข้อมูล (Chunks)" โดยทั่วไปหมายถึงส่วนของข้อมูล*ภายนอก* ที่ใช้ในระบบอย่างเช่น Retrieval-Augmented Generation (RAG) [27] RAG เป็นระบบ*เสริม*ที่ช่วยให้ LLM เข้าถึงข้อมูลเฉพาะทาง ข้อมูลล่าสุด หรือข้อมูลส่วนตัว ระบบ RAG จะดึงชิ้นส่วนข้อความที่เกี่ยวข้องจากฐานข้อมูลภายนอก (ซึ่งมักจะเป็นฐานข้อมูลเวกเตอร์ที่ชิ้นส่วนข้อมูลเหล่านั้นถูกแปลงเป็นเอ็มเบดดิ้งเช่นกัน) และชิ้นส่วนข้อความเหล่านี้จะถูกป้อน*เข้าไป*ในคำสั่งของ LLM พร้อมกับคำถามเดิม
- ดังนั้น ตัว LLM เองไม่ได้ "ค้นหาชิ้นส่วนข้อมูล" ภายในกลไกพื้นฐานของมัน มันประมวลผลเวกเตอร์ หากมีการใช้ RAG ชิ้นส่วนข้อความจะถูก*ส่งมอบ*ให้กับ LLM เป็นส่วนหนึ่งของข้อมูลนำเข้า

กลไก Attention (QKV) นี่เองที่เป็นกระบวนการ "ค้นหา" และ "ดึงข้อมูล" ภายในของ LLM มันไม่ได้ค้นหาข้อเท็จจริงจากฐานข้อมูลเชิงสัญลักษณ์ แต่เป็นการคำนวณความเกี่ยวข้องระหว่างส่วนต่างๆ ของข้อมูลนำเข้า (และสถานะภายในของมันเอง) ที่ถูกแทนด้วยเวกเตอร์อย่างต่อเนื่อง นี่คือการเปลี่ยนแปลงพื้นฐานจากวิธีการค้นหาแบบดั้งเดิม ข้อมูลที่ "ค้นพบ" หรือที่ถูกให้ความสนใจนั้นจะอยู่ในรูปของเวกเตอร์เสมอ (โดยเฉพาะอย่างยิ่งคือผลรวมถ่วงน้ำหนักของ Value เวกเตอร์) ซึ่งจะถูกประมวลผลโดยชั้นต่อๆ ไป LLM ไม่เคย "ดึง" ชิ้นส่วนข้อความที่ไม่ต่อเนื่องออกมาจากค่าน้ำหนักภายในของมัน เหมือนกับที่ RAG ดึงชิ้นส่วนข้อความจากฐานข้อมูลภายนอก

การมี Multi-Head Attention [5] ช่วยให้ LLM สามารถพิจารณาความสัมพันธ์ประเภทต่างๆ (เช่น ไวยากรณ์ ความหมาย การอ้างอิง) ระหว่างโทเค็นได้พร้อมกัน การประมวลผล "มุมมอง" ที่แตกต่างกันเหล่านี้แบบขนานเป็นกุญแจสำคัญสู่ความเข้าใจบริบทที่ซับซ้อนของมัน สถานะซ่อนเร้น [24] ไม่ใช่แค่การแทนข้อมูลแบบคงที่ แต่มีการเปลี่ยนแปลงและพัฒนาเมื่อข้อมูลไหลผ่านชั้นต่างๆ แต่ละชั้นจะปรับปรุงเวกเตอร์บริบทตามสิ่งที่กลไก attention ในชั้นนั้นเห็นว่าสำคัญ การประมวลผลแบบลำดับชั้นนี้ช่วยให้เกิดความเข้าใจที่เป็นนามธรรมมากขึ้นเรื่อยๆ

การแยกแยะความแตกต่างระหว่างการประมวลผลเวกเตอร์ภายในของ LLM กับการจัดการชิ้นส่วนข้อมูลภายนอก (เช่นใน RAG [27]) เป็นสิ่งสำคัญอย่างยิ่งสำหรับการสอนที่ถูกต้อง นักเรียนจำเป็นต้องเข้าใจว่า LLM อย่าง ChatGPT โดยพื้นฐานแล้วไม่ได้ "ค้นหาเว็บ" หรือ "ค้นหาไฟล์" เว้นแต่จะได้รับการออกแบบมาเป็นพิเศษให้เป็น agent ที่มีเครื่องมือ หรือใช้

ระบบ RAG

ตาราง: การ "ค้นหา" ภายในของ LLM เทียบกับการค้นหาทั่วไป (เช่น Google)

คุณลักษณะ	การประมวลผลภายในของ LLM (ผ่าน Attention)	เครื่องมือค้นหาทั่วไป (เช่น Google)
สิ่งที่ "ค้นหา"	ความเข้าใจของตัวเองเกี่ยวกับคำสั่งที่ป้อนเข้ามา และรูปแบบที่เรียนรู้มา (ค่าน้ำหนัก)	หน้าเว็บ เอกสาร ฯลฯ หลายพันล้านรายการในดัชนี
วิธี "ค้นหา" ข้อมูล	คำนวณความเกี่ยวข้องระหว่างส่วนต่างๆ ของข้อความ (ในรูปเวกเตอร์) โดยใช้ Q, K, V	จับคู่คำสำคัญ ลิงก์ และสัญญาณอื่นๆ กับคำค้นหา
สิ่งที่ "ดึงข้อมูล" ออกมา	การแทนข้อมูลด้วยเวกเตอร์ที่สมบูรณ์ด้วยบริบท (สถานะซ่อนเร้นใหม่)	ลิงก์ไปยังหน้าเว็บ ส่วนย่อยของข้อความ คำตอบโดยตรง
ข้อมูลพื้นฐาน	ค่าน้ำหนักที่เป็นตัวเลข และการแทนข้อมูลด้วยเวกเตอร์	ข้อความ รูปภาพ วิดีโอ ที่จัดทำดัชนีไว้

ตารางนี้ช่วยตอบคำถามโดยนัยของผู้ใช้เกี่ยวกับวิธีที่ LLM "ค้นหา" โดยเปรียบเทียบกระบวนการภายในที่อิงกับคณิตศาสตร์และเวกเตอร์ กับแนวคิดที่คุ้นเคยกว่าของการค้นหาเว็บ ซึ่งช่วยเน้นความแตกต่างพื้นฐานและป้องกันความเข้าใจผิด มันชี้แจงว่า "การค้นหา" ใน LLM เป็นการเปรียบเทียบสำหรับกระบวนการคำนวณที่ซับซ้อน

5. ขั้นตอนที่ 4: รังสรรค์คำตอบ – การสร้างประโยคและบทความ

หลังจากที่ LLM ประมวลผลคำสั่งและ "ทำความเข้าใจ" บริบท (ซึ่งถูกแทนด้วยเวกเตอร์สถานะซ่อนเร้นสุดท้าย) แล้ว มันก็จะเริ่มสร้างคำตอบออกมา กระบวนการนี้เรียกว่า "การสร้างแบบอัตตสืบเนื่อง" (Autoregressive Generation) ซึ่งหมายความว่า LLM จะสร้างโทเค็น (คำหรือส่วนของคำ) ออกมาทีละตัว [7]

ทีละคำ: การสร้างแบบอัตตสืบเนื่อง (Autoregressive Generation)

LLM สร้างข้อความตอบสนองโดยการทำนายโทเค็นถัดไปทีละโทเค็น [10] เมื่อโทเค็นใหม่ถูกสร้างขึ้น มันจะถูกเพิ่มเข้าไปในลำดับของข้อความที่สร้างขึ้นแล้ว และลำดับที่ปรับปรุงใหม่นี้จะถูกใช้เพื่อทำนายโทเค็น *ถัดไป* อีกที เป็นวงจรเช่นนี้ไปเรื่อยๆ ลองนึกภาพว่ามันเหมือนกับการ "พยายามพูดประโยคให้จบทีละคำ โดยแต่ละคำที่คุณเลือกจะขึ้นอยู่กับคำที่คุณพูดไปแล้วก่อนหน้านี้" [1] หรือเหมือน "นักเล่านิทานที่ตัดสินใจเลือกคำถัดไปของเรื่องราวโดยอิงจากทุกสิ่งที่เล่าไปแล้ว"

ความมหัศจรรย์ของการทำนาย: ความน่าจะเป็นและทางเลือก

ในแต่ละขั้นตอน LLM ไม่ได้เลือกคำเพียงคำเดียว แต่มันคำนวณ "คะแนนความน่าจะเป็น" (probability score) สำหรับ

ทุกโทเค็นที่เป็นไปได้ในชุดคำศัพท์ของมัน ว่าโทเค็นใดควรจะมาเป็นลำดับถัดไป [7] จากนั้น LLM จะต้องเลือกโทเค็นจริงจากความน่าจะเป็นเหล่านี้ ซึ่งมีหลายวิธีในการเลือก หรือที่เรียกว่า "กลยุทธ์การถอดรหัส" (Decoding Strategies):

- การค้นหาแบบละโมบ (Greedy Search): เลือกโทเค็นที่มีความน่าจะเป็นสูงสุดเสมอ วิธีนี้รวดเร็ว แต่อาจทำให้ได้ข้อความที่ซ้ำซากหรือน่าเบื่อ [7]
- การค้นหาแบบบีม (Beam Search): ติดตามลำดับของโทเค็นที่มีความน่าจะเป็นสูงหลายๆ ลำดับพร้อมกัน แล้วเลือกเอาลำดับที่ดีที่สุดโดยรวม ทำให้ได้ข้อความที่สอดคล้องกันมากขึ้น แต่ก็ช้าลง [7]
- การสุ่มตัวอย่าง (Sampling - Top-k, Nucleus/Top-p): เพิ่มความสุ่มโดยการเลือกจากกลุ่มย่อยของโทเค็นที่มีแนวโน้มมากที่สุด วิธีนี้ทำให้คำตอบมีความคิดสร้างสรรค์และหลากหลายมากขึ้น [7] "อุณหภูมิ" (Temperature) เป็นพารามิเตอร์ที่ใช้ควบคุมระดับความสุ่มนี้

การสร้างข้อความที่สอดคล้องกัน

ด้วยการทำนายโทเค็นถัดไปที่น่าจะเป็นที่สุดซ้ำๆ โดยอ้างอิงจากคำสั่งและข้อความที่สร้างขึ้นแล้ว LLM จะร้อยเรียงโทเค็นเข้าด้วยกันเพื่อสร้างเป็นประโยค จากนั้นเป็นย่อหน้า และในที่สุดก็เป็นบทความที่ยาวขึ้น กลไก Attention ยังคงมีบทบาทสำคัญในขั้นตอนนี้ โดยช่วยให้ LLM จดจ่ออยู่กับส่วนที่เกี่ยวข้องของคำสั่งและสิ่งที่มันได้เขียนไปแล้ว

รู้ว่าเมื่อใดควรหยุด

LLM จำเป็นต้องมีสัญญาณเพื่อหยุดการสร้างข้อความ ซึ่งอาจเป็น:

- การสร้างโทเค็นพิเศษที่เรียกว่า "โทเค็นสิ้นสุดลำดับ" (end-of-sequence - EOS token) [31]
- การสร้างข้อความจนถึงความยาวสูงสุดของโทเค็นที่กำหนดไว้ล่วงหน้า [31]

การสร้างข้อความนั้นมีลักษณะเป็นความน่าจะเป็นโดยพื้นฐาน LLM ไม่ได้ "ตัดสินใจ" ในความหมายแบบมนุษย์ แต่เป็นการสุ่มตัวอย่างจากชุดการแจกแจงของโทเค็นถัดไปที่น่าจะเป็น สิ่งนี้อธิบายได้ว่าทำไมเราถึงอาจได้รับคำตอบที่แตกต่างกันเล็กน้อยสำหรับคำสั่งเดียวกัน [7] การเลือกกลยุทธ์การถอดรหัส [7] ส่งผลโดยตรงต่อ "ความคิดสร้างสรรค์" ความสอดคล้อง และความซ้ำซากของผลลัพธ์ที่ LLM สร้างขึ้น เนื่องจากเป็นการสร้างแบบอัตตสืบเนื่องและอิงตามความน่าจะเป็น หากแบบจำลองเริ่มเดินไปในทิศทางของโทเค็นที่ไม่ถูกต้องเล็กน้อยแต่ยังคงมีความน่าจะเป็นสูง มันสามารถสะสมข้อผิดพลาดเหล่านี้และ "หลอน" หรือสร้างข้อความที่ไม่มีความหมายได้ มันพยายามทำนายสิ่งที่สอดคล้องในระดับท้องถิ่นอยู่เสมอ [10] การสร้างข้อความยาวๆ ทีละโทเค็นนั้นต้องใช้การคำนวณสูง การทำนายแต่ละโทเค็นจำเป็นต้องผ่าน (อย่างน้อยส่วนหนึ่งของ) แบบจำลองทั้งหมด [10]

6. สรุป: การเดินทางของ LLM – จากคำสั่งสู่ข้อความ

ถึงตอนนี้ เราได้เดินทางผ่านกระบวนการทำงานภายในของ LLM ตั้งแต่การรับคำสั่งไปจนถึงการสร้างคำตอบออกมาเป็นข้อความแล้ว ลองมาทบทวนขั้นตอนสำคัญๆ กันอีกครั้ง

สรุปย่อเพื่อความเข้าใจ!

1. คำสั่ง (Prompt) ของเราถูก "หั่น" เป็นชิ้นเล็กๆ เรียกว่า โทเค็น (Tokens)
2. โทเค็นเหล่านี้จะได้รับ "รหัสลับ" (คือ เอ็มเบดดิ้ง (Embeddings) ที่เป็นชุดตัวเลข) และ "หมายเลขตำแหน่ง" เพื่อบอกลำดับ
3. รหัสเหล่านี้จะถูกส่งเข้าไปใน "สมอง" ของ LLM ซึ่งก็คือ Transformer
4. กลไก Attention จะทำหน้าที่เหมือน "สปอตไลท์อัจฉริยะ" คอยค้นหาและให้ความสำคัญกับข้อมูลที่เกี่ยวข้อง
5. ข้อมูลจะถูกแปลงเป็น "รหัสเวกเตอร์" ชุดใหม่ที่เรียกว่า สถานะซ่อนเร้น (Hidden States) ซึ่งเป็นเหมือน "ความคิด" ที่กำลังพัฒนาของ LLM
6. สุดท้าย LLM จะ "ทาย" คำตอบออกมาทีละคำ (โทเค็น) ด้วยกระบวนการ การสร้างแบบอัตตสืบเนื่อง (Autoregressive Generation)

LLM คืออะไร (และไม่ใช่อะไร): สิ่งสำคัญที่ต้องย้ำคือ LLM เป็นเครื่องมือจับคู่รูปแบบและทำนายที่ก้าวหน้าอย่างยิ่ง [1] พวกมันเรียนรู้จากข้อมูลจำนวนมหาศาล แต่ไม่ได้ "เข้าใจ" หรือ "รู้สึก" เหมือนมนุษย์ ไม่ได้มีความเชื่อหรือเจตนาเป็นของตัวเอง "ความรู้" ของพวกมันขึ้นอยู่กับความสัมพันธ์ทางสถิติในข้อมูลที่ใช้ฝึกฝน

ทำไมสิ่งนี้จึงสำคัญสำหรับนักเรียน:

- การใช้งานที่ดีขึ้น: การเข้าใจวิธีการทำงานของ LLM ช่วยให้เราใช้งานมันได้อย่างมีประสิทธิภาพมากขึ้น รู้ว่าควรตั้งคำถามอย่างไรเพื่อให้ได้คำตอบที่ดีที่สุด
- การประเมินผลลัพธ์อย่างมีวิจารณญาณ: เมื่อรู้ว่า LLM อาจทำผิดพลาดหรือ "หลอน" ได้ (สร้างข้อมูลที่ไม่ถูกต้อง) เราก็จะระมัดระวังและตรวจสอบข้อมูลที่ได้จากมันมากขึ้น
- ก้าวทันอนาคต: เทคโนโลยีนี้กำลังเข้ามามีบทบาทในชีวิตประจำวันมากขึ้นเรื่อยๆ การมีความรู้พื้นฐานเกี่ยวกับ LLM ก็เหมือนกับการรู้ว่าเครื่องยนต์รถยนต์ทำงานอย่างไร (ในระดับหนึ่ง!) ซึ่งเป็นทักษะที่สำคัญในโลกยุคใหม่

แม้ว่าเราจะสามารถอธิบายกลไกพื้นฐานเหล่านี้ได้ แต่ความสามารถอันน่าทึ่งของ LLM (เช่น การเขียนโค้ด การสรุปความ หรือการให้เหตุผลที่ซับซ้อนในระดับหนึ่ง) มักจะ "ปรากฏขึ้นเอง" (emergent abilities) จากกระบวนการที่ค่อนข้างเรียบง่ายเหล่านี้ เมื่อถูกนำไปใช้กับข้อมูลและขนาดของแบบจำลองที่ใหญ่โตมโหฬาร [33] นี่เป็นแง่มุมที่น่าสนใจอย่างยิ่ง – พฤติกรรมที่ซับซ้อนเกิดขึ้นจากกฎเกณฑ์ที่ง่ายกว่า การทำความเข้าใจธรรมชาติของ LLM ที่ขับเคลื่อนด้วยข้อมูลและความน่าจะเป็นนี้ เป็นก้าวแรกที่สำคัญในการอภิปรายถึงผลกระทบทางจริยธรรม ความลำเอียงที่อาจมีอยู่ในข้อมูลที่ใช้ฝึกฝน และศักยภาพในการนำไปใช้ในทางที่ผิด ซึ่งเป็นสิ่งที่สำคัญอย่างยิ่งสำหรับนักเรียนในฐานะผู้ใช้งาน AI ในอนาคต

7. อภิธานศัพท์ฉบับย่อ (เครื่องมือสำหรับนำเสนอผลงานของคุณ!)

ต่อไปนี้คือคำศัพท์สำคัญพร้อมคำอธิบายง่ายๆ ที่เป็นมิตรกับนักเรียน:

- LLM (Large Language Model - แบบจำลองภาษาขนาดใหญ่): AI ทรงพลังที่เข้าใจและสร้างข้อความคล้ายมนุษย์ โดยเรียนรู้รูปแบบจากข้อมูลจำนวนมหาศาล [2]

- Prompt (คำสั่ง): คำถามหรือคำสั่งที่คุณป้อนให้กับ LLM [36]
- Tokenization (การแบ่งเป็นโทเค็น): การแบ่งข้อความออกเป็นชิ้นส่วนเล็กๆ (คำหรือส่วนของคำ) ที่ LLM สามารถประมวลผลได้ [2]
- Embedding (เอ็มเบดดิ้ง): การเปลี่ยนโทเค็นให้เป็นชุดของตัวเลข (เวกเตอร์) ที่แทนความหมายของมัน คำที่มีความหมายคล้ายกันจะได้รหัสตัวเลขที่คล้ายกัน [2]
- Vector (เวกเตอร์): ชุดของตัวเลข ใน LLM เวกเตอร์ใช้แทนความหมายหรือคุณลักษณะของโทเค็น
- Transformer (ทรานส์ฟอร์มเมอร์): "สมอง" หลัก (สถาปัตยกรรม) ที่ใช้ใน LLM สมัยใหม่ มีชื่อเสียงด้านกลไก Attention [2]
- Attention (Self-Attention - การใส่ใจ/การใส่ใจตนเอง): ความสามารถของ LLM ในการมุ่งเน้นไปที่คำหรือโทเค็นที่สำคัญที่สุดในข้อความเพื่อทำความเข้าใจบริบทและความหมาย [2]
- Query, Key, Value (QKV - คิวรี, คีย์, แวลู): บทบาทสามอย่างที่โทเค็นใช้ในกลไก Attention เพื่อคำนวณความเกี่ยวข้อง [12]
- Hidden State (สถานะซ่อนเร้น): การแทนข้อมูลด้วยตัวเลข (เวกเตอร์) ขณะที่ข้อมูลกำลังถูกประมวลผลผ่านชั้นต่างๆ ของ LLM เปรียบเหมือน "ความคิด" ที่กำลังพัฒนาของ LLM เกี่ยวกับข้อมูลนำเข้า [23]
- Autoregressive Generation (การสร้างแบบอัตตสืบเนื่อง): การสร้างข้อความทีละโทเค็น (คำหรือส่วนของคำ) โดยแต่ละโทเค็นใหม่จะขึ้นอยู่กับโทเค็นก่อนหน้า [10]
- Hallucination (การหลอน): เมื่อ LLM สร้างข้อความที่ฟังดูสมเหตุสมผลแต่ไม่ถูกต้องหรือไม่มีความหมาย [1]

ตาราง: แนวคิดหลักและอุปมาอุปมัย

แนวคิด LLM	คำอธิบายง่ายๆ	อุปมาอุปมัยที่ใช้
LLM	AI ที่เข้าใจและสร้างข้อความโดยเรียนรู้รูปแบบ	สุดยอดระบบเติมคำอัตโนมัติ, นกแก้วผู้รอบรู้
Tokenization	การแบ่งข้อความออกเป็นชิ้นเล็กๆ	หั่นส่วนผสม, ตัวต่อเลโก้
Embedding	การเปลี่ยนโทเค็นเป็นรหัสตัวเลข (เวกเตอร์) ที่แทนความหมาย	พิกัดบน "แผนที่ความหมาย", รหัสลับ
Positional Encoding	การเพิ่มข้อมูลเกี่ยวกับลำดับของคำ	การใส่หมายเลขกำกับขั้นตอนในสูตรอาหาร, การประทับเวลา
Transformer Architecture	"สมอง" หรือเครื่องยนต์ของ LLM	โรงงานที่มีสายการผลิต, เครื่องจักรที่ซับซ้อน
Attention (QKV)	วิธีที่ LLM มุ่งเน้นไปที่ข้อมูลสำคัญ	บรรณารักษ์ค้นหาหนังสือ, สปอตไลท์, การจับคู่

Hidden States	"บทสรุป" เชิงตัวเลขที่กำลังพัฒนาของ LLM เกี่ยวกับข้อมูลนำเข้า	ความคิดที่กำลังพัฒนา, ขั้นตอนการคำนวณระหว่างทาง
Autoregressive Generation	การสร้างข้อความทีละคำ	การเติมประโยคให้สมบูรณ์, การเล่านิทานทีละคำ
Hallucination	LLM สร้างเรื่องขึ้นเองอย่างมั่นใจ	นกแก้วพูดข้อมูลผิดๆ, ผู้พูดผิดพลาดอย่างมั่นใจ

ผลงานที่อ้างอิง

1. How to explain LLM to an idiot? : r/ChatGPT - Reddit, เข้าถึงเมื่อ มิถุนายน 13, 2025 https://www.reddit.com/r/ChatGPT/comments/1gpz0m1/how_to_explain_llm_to_an_idiot/
2. LLM Glossary - Civitai Education, เข้าถึงเมื่อ มิถุนายน 13, 2025 https://education.civitai.com/llm-glossary/
3. Tokenization vs Embedding - How are they Different? | Airbyte, เข้าถึงเมื่อ มิถุนายน 13, 2025 https://airbyte.com/data-engineering-resources/tokenization-vs-embeddings
4. confused about embeddings and tokenization in LLMs : r/learnmachinelearning - Reddit, เข้าถึงเมื่อ มิถุนายน 13, 2025 https://www.reddit.com/r/learnmachinelearning/comments/1cs29kn/confused_about_embeddings_and_tokenization_in_llms/
5. A Beginner's Guide to the Transformer Architecture in Deep ..., เข้าถึงเมื่อ มิถุนายน 13, 2025 https://compute.hivenet.com/post/simplifying-transformer-architecture-a-beginners-guide-to-understanding-ai-magic
6. Tokenization in NLP - GeeksforGeeks, เข้าถึงเมื่อ มิถุนายน 13, 2025 https://www.geeksforgeeks.org/nlp-how-tokenizing-text-sentence-words-works/
7. Decoding Strategies in Large Language Models – Maxime Labonne, เข้าถึงเมื่อ มิถุนายน 13, 2025 https://mlabonne.github.io/blog/posts/2023-06-07-Decoding_strategies.html
8. 11.1 Vector embeddings and tokens - 6.390, เข้าถึงเมื่อ มิถุนายน 13, 2025 https://introml.mit.edu/_static/fall23/LectureNotes/chapter_Transformers_and_large_language_models.pdf
9. Visualising Word2Vec Word Vectors, เข้าถึงเมื่อ มิถุนายน 13, 2025 https://rlbarter.github.io/superheat-examples/word2vec/
10. How Large Language Model Generate Text, เข้าถึงเมื่อ มิถุนายน 13, 2025

https://kolosal.ai/articles/LLM-Optimization/How-LLM-Generate-Tokens
11. The Annotated Transformer - Harvard NLP, เข้าถึงเมื่อ มิถุนายน 13, 2025 https://nlp.seas.harvard.edu/annotated-transformer/
12. Explainable AI: Visualizing Attention in Transformers - Comet.ml, เข้าถึงเมื่อ มิถุนายน 13, 2025 https://www.comet.com/site/blog/explainable-ai-for-transformers/
13. The Illustrated Transformer From Scratch, เข้าถึงเมื่อ มิถุนายน 13, 2025 https://idtjo.hosting.acm.org/wordpress/the-illustrated-transformer/
14. An Intuitive Introduction to Transformers - Paperspace Blog, เข้าถึงเมื่อ มิถุนายน 13, 2025 https://blog.paperspace.com/attention-is-all-you-need-the-components-of-the-transformer/
15. Transformer Attention Mechanism in NLP - GeeksforGeeks, เข้าถึงเมื่อ มิถุนายน 13, 2025 https://www.geeksforgeeks.org/transformer-attention-mechanism-in-nlp/
16. Understanding the Transformer architecture for neural networks, เข้าถึงเมื่อ มิถุนายน 13, 2025 https://www.jeremyjordan.me/transformer-architecture/
17. Transformer Attention: A Guide to the Q, K, and V Matrices - billparker.ai, เข้าถึงเมื่อ มิถุนายน 13, 2025 https://www.billparker.ai/2024/10/transformer-attention-simple-guide-to-q.html
18. [D] How to truly understand attention mechanism in transformers? : r/MachineLearning - Reddit, เข้าถึงเมื่อ มิถุนายน 13, 2025 https://www.reddit.com/r/MachineLearning/comments/qidpqx/d_how_to_truly_understand_attention_mechanism_in/
19. Understanding Attention Mechanism in Transformer Neural Networks - Learn OpenCV, เข้าถึงเมื่อ มิถุนายน 13, 2025 https://learnopencv.com/attention-mechanism-in-transformer-neural-networks/
20. An Intuition for Attention - Jay Mody, เข้าถึงเมื่อ มิถุนายน 13, 2025 https://jaykmody.com/blog/attention-intuition/
21. [D] How does 'self-attention' work in transformer models? : r/MLQuestions - Reddit, เข้าถึงเมื่อ มิถุนายน 13, 2025 https://www.reddit.com/r/MLQuestions/comments/16q8nkr/d_how_does_selfattention_work_in_transformer/
22. raw.githubusercontent.com, เข้าถึงเมื่อ มิถุนายน 13, 2025 https://raw.githubusercontent.com/mittapallynitin/blogs/refs/heads/main/attention-transformers.md
23. Reasoning Models Know When They're Right: Probing Hidden States for Self-Verification,

เข้าถึงเมื่อ มิถุนายน 13, 2025 https://arxiv.org/html/2504.05419v1
24. Writing an LLM from scratch, part 5 -- more on self-attention :: Giles ..., เข้าถึงเมื่อ มิถุนายน 13, 2025 https://www.gilesthomas.com/2025/01/llm-from-scratch-5-self-attention
25. Understanding the Hidden Layers in Large Language Models LLMs - MarkTechPost, เข้าถึงเมื่อ มิถุนายน 13, 2025 https://www.marktechpost.com/2024/09/11/understanding-the-hidden-layers-in-large-language-models-llms/
26. Understanding the hidden state of the middle layer of the Transformer model - Reddit, เข้าถึงเมื่อ มิถุนายน 13, 2025 https://www.reddit.com/r/LanguageTechnology/comments/1bigdzq/understanding_the_hidden_state_of_the_middle/
27. Practical Guide to LLM Chunking: Context, RAG, Vectors - Mindee, เข้าถึงเมื่อ มิถุนายน 13, 2025 https://www.mindee.com/blog/llm-chunking-strategies
28. Retrieval Augmented Generation for Beginners - PromptHub, เข้าถึงเมื่อ มิถุนายน 13, 2025 https://www.prompthub.us/blog/retrieval-augmented-generation-for-beginners
29. Retrieval Augmented Generation (RAG) | Pinecone, เข้าถึงเมื่อ มิถุนายน 13, 2025 https://www.pinecone.io/learn/retrieval-augmented-generation/
30. Are LLMs And Search Engines The Same? - Search Engine Journal, เข้าถึงเมื่อ มิถุนายน 13, 2025 https://www.searchenginejournal.com/are-llms-and-search-engines-the-same/500057/
31. Generation with LLMs - Hugging Face, เข้าถึงเมื่อ มิถุนายน 13, 2025 https://huggingface.co/docs/transformers/v4.34.1/llm_tutorial
32. GenAI: LLM Decoding Strategies Explained | Greedy, Beam, Top-k, Top-p, Temperature, Contrastive - YouTube, เข้าถึงเมื่อ มิถุนายน 13, 2025 https://www.youtube.com/watch?v=JqgsxC4guK4
33. [R] Are Emergent Abilities in Large Language Models just In-Context Learning? - Reddit, เข้าถึงเมื่อ มิถุนายน 13, 2025 https://www.reddit.com/r/MachineLearning/comments/19bkcqz/r_are_emergent_abilities_in_large_language_models/
34. Emergent Abilities of Large Language Models - AssemblyAI, เข้าถึงเมื่อ มิถุนายน 13, 2025 https://www.assemblyai.com/blog/emergent-abilities-of-large-language-models
35. LLM terminology. The top 50 terms to know - Nebuly, เข้าถึงเมื่อ มิถุนายน 13, 2025 https://www.nebuly.com/blog/llm-terminology-the-top-50-terms-to-know

36. LLM Prompting: How to Prompt LLMs for Best Results - Multimodal, เข้าถึงเมื่อ มิถุนายน 13, 2025 https://www.multimodal.dev/post/llm-prompting
37. LLM prompt engineering | Tutorial for developers, เข้าถึงเมื่อ มิถุนายน 13, 2025 https://pieces.app/blog/llm-prompt-engineering